

ReAct vs Tree-of-Thoughts: the Cost of Search, by Hand

Why exploring a *tree* of thoughts costs a geometric series of LLM calls — every call counted, for one small reasoning problem.

What this document does. It takes the two planning patterns from Module 2.10 that sit at opposite ends of the cost axis — **ReAct** (cost ★□□□) and **Tree of Thoughts** (cost ★★☆☆) — and counts the LLM calls each one makes on the *same* small task: a hard reasoning step (think Game-of-24) where ReAct walks one line and ToT explores a branching tree. We fix the branching factor b and the depth D , derive the node count as a geometric series *by hand*, and then price it. Every number below is reproduced by the Python program in Appendix A, so the arithmetic is exact. The module says ToT runs “10–20×” the cost of ReAct; we rebuild that multiplier from first principles and show exactly where it comes from — and what beam-pruning does to it.

Where this sits. This is **Topic 1 of Module 2.10** (Planning Patterns) — the quantitative spine under the cost-star ratings in the pattern quadrant. Its companions: Module 0.0 Topic 1 (the cost of the ReAct loop — the linear baseline this builds on) and Module 2.10 Topic 2 (Reflexion’s attempt scaling — the *other* way to spend calls buying reliability).

Concepts covered, in order: ReAct as a single linear path · a thought-tree of branching b and depth D · the geometric series $\frac{b^{D+1}-1}{b-1}$ (derived) · generate + evaluate calls per node · the combinatorial explosion · beam pruning to $w b D$ calls · calls $\rightarrow$ dollars · why ToT is a research tool, not an online default.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one hard step, two ways to attack it

A **ReAct** agent walks a *single path*. Each turn it thinks, acts, observes, and commits — there is no going back, no second candidate. To take K steps it makes K LLM decision calls. That is the whole cost: linear in the number of steps.

A **Tree-of-Thoughts** agent refuses to commit. At every node it *generates* b candidate next-thoughts, *evaluates* each one with a scoring call, and recurses. Instead of a line it grows a tree. The question this document answers is: how many calls does that tree cost, and how does pruning tame it?

```
1 # ReAct: one path, K calls
2 def react(goal, K):
3     for step in range(K):
4         thought, action = llm.decide(goal, history) # 1 call/step
5         # -> K calls total, linear
6
7 # Tree of Thoughts: branch b, depth D
8 def tot(node, depth):
9     if depth == 0: return score(node)
10    children = llm.generate(node, b) # 1 generate call -> b children
11    for c in children:
12        llm.evaluate(c) # 1 score call each
13        tot(c, depth - 1) # recurse -> tree
```

The quantities. We give the problem round, hand-followable parameters. Realism is secondary to clarity — the mechanics are identical at any b, D .

Quantity	Symbol	Value here
Branching factor (candidates per node)	b	3
Search depth (levels below the root)	D	3
ReAct path length (steps)	K	5
Beam width (nodes kept per level)	w	2
Tokens read per call (flat, illustrative)	T_{in}	2,000
Tokens written per call	T_{out}	300
Input price	p_{in}	\$3 / 1M tokens
Output price	p_{out}	\$15 / 1M tokens

Notation. We count *LLM calls* first — that is the pattern-independent currency — then attach a flat dollar figure per call ($c_{\text{call}} = T_{\text{in}} p_{\text{in}} + T_{\text{out}} p_{\text{out}}$) so the two patterns are comparable. A “node” costs one generate call and one evaluate call.

Intuition

ReAct is a person walking down a corridor, opening one door per step. Tree-of-Thoughts is a person who, at every door, clones themselves to try all b doors at once — and each clone clones again at the next door. After D doors you do not have D people; you have b^D of them. The bill is not the depth you reach; it is the *tree you grew to reach it*.

2 Step 1 — ReAct is a straight line

There is almost nothing to derive here, and that is the point. ReAct keeps one trajectory. Step 1 is one decision call; step 2 is one more; and so on. After K steps:

$$\text{calls}_{\text{ReAct}}(K) = K. \quad (1)$$

For our hard problem we let ReAct take $K = 5$ steps before it answers (or gives up). That is **5 calls**. Linear, cheap, and — on a genuine search problem where the first line of reasoning is usually wrong — often *wrong*, because ReAct never explores an alternative. That trade is exactly what ToT is buying back, at the price we now compute.

3 Step 2 — the thought-tree, and its geometric series

Plant the root at the top (depth 0). It generates b children (depth 1). Each of those generates b more (depth 2). The number of nodes at level ℓ is b^ℓ , so the total node count down to depth D is the sum

$$N(b, D) = \sum_{\ell=0}^D b^\ell = 1 + b + b^2 + \cdots + b^D. \quad (2)$$

This is a finite **geometric series**, and it has a closed form. Multiply N by b and subtract — the classic telescoping trick:

$$\begin{aligned}
bN &= b + b^2 + \dots + b^D + b^{D+1}, \\
bN - N &= b^{D+1} - 1 \quad (\text{every middle term cancels}), \\
N(b, D) &= \boxed{\frac{b^{D+1} - 1}{b - 1}}. \tag{3}
\end{aligned}$$

Every node needs one *generate* call (to produce its children) and one *evaluate* call (to score it), so the unpruned call count is about $2N$.

Mini-example — this operation in isolation

Take $b = 3$, $D = 3$. The series is $1 + 3 + 9 + 27$. Add by hand: $1 + 3 = 4$, $+9 = 13$, $+27 = 40$. The closed form agrees: $\frac{3^4-1}{3-1} = \frac{81-1}{2} = \frac{80}{2} = 40$. ✓ So the tree has **40 nodes** — 40 generate calls, and ~ 40 evaluate calls on top. Against ReAct’s **5**, that is the explosion in one number: forty versus five, before you have paid for a single score.

4 Step 3 — the explosion, tabulated

The geometric series grows like b^D — it is exponential in depth. A flat table makes the shape impossible to miss. “ToT-unpruned” is the gen+evaluate count $2N(b, D)$; ReAct takes $K = D+2$ steps (a path a little longer than the tree is deep):

b	D	ReAct calls	ToT unpruned ($2N$)	ratio
2	2	4	14	$3.5\times$
2	3	5	30	$6\times$
3	2	4	26	$6.5\times$
3	3	5	80	$16\times$
3	4	6	242	$40\times$
4	3	5	170	$34\times$

At $b = 3$, $D = 3$ the ratio is $80/5 = 16\times$ — squarely inside the module’s quoted “10–20 $\times$ ” band, and we got there with nothing but the geometric series. Push the depth one notch to $D = 4$ and it is $40\times$; the bill does not creep, it leaps, because b^D leaps. Sample check, $b = 3$, $D = 4$: $N = \frac{3^5-1}{2} = \frac{242}{2} = 121$, so $2N = 242$. ✓

Watch out

The pitfall the module calls *branch explosion* is this row read one line further: $b = 8$, $D = 4$ gives $N = \frac{8^5-1}{7} = 4681$ nodes — thousands of calls for one query. The cure is not a faster model; it is to stop generating the whole tree. That is beam pruning, next.

5 Step 4 — beam pruning brings it back to earth

The fix is to refuse to keep the whole frontier. With **beam width** w , after scoring a level you keep only the top w nodes and discard the rest. The tree never fans out past w live nodes. At

each of the D levels you expand those w kept nodes into $w b$ children and score them, so the call count is linear again:

$$\boxed{\text{calls}_{\text{beam}}(w, b, D) \approx w b D} \quad (4)$$

For $w = 2, b = 3, D = 3$ that is $2 \cdot 3 \cdot 3 = \mathbf{18}$ calls — versus 80 unpruned and 5 for ReAct. Pruning collapsed the exponential b^D back to a *product* $w b D$: still more than ReAct, but a constant multiple, not an explosion.

b	D	ReAct	ToT unpruned	ToT beam ($w=2$)
2	2	4	14	8
2	3	5	30	12
3	2	4	26	12
3	3	5	80	18
3	4	6	242	24
4	3	5	170	24

Notice what beam search traded away: it is no longer a complete search. You keep only w lines, so if the winning thought scored 4th at some level, you have already thrown it out. **Beam buys back the cost by spending reliability** — the very thing ToT existed to buy. That tension is the whole engineering problem of search agents.

6 Step 5 — calls into dollars

Calls are the honest currency, but the module rates these patterns by cost stars, so let us close the loop in dollars. A flat call reads $T_{\text{in}} = 2000$ and writes $T_{\text{out}} = 300$ tokens:

$$c_{\text{call}} = 2000 \cdot 3 \times 10^{-6} + 300 \cdot 15 \times 10^{-6} = 0.0060 + 0.0045 = \$0.0105.$$

Multiply the call counts at our headline $b = 3, D = 3$:

Pattern	calls	cost	vs. ReAct
ReAct ($K = 5$)	5	\$0.0525	1×
ToT beam ($w = 2$)	18	\$0.1890	3.6×
ToT unpruned	80	\$0.8400	16×

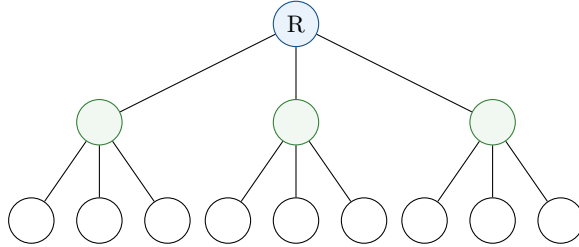
Sample check, unpruned: $80 \times \$0.0105 = \0.84 . ✓ The unpruned tree costs 16× a single ReAct pass for *one* query; beam search shaves that to 3.6×. Whether either is worth it depends entirely on one thing: does a single ReAct pass actually fail on this task often enough to justify paying 4–16× to search?

Intuition

Search is insurance, and like all insurance it only pays off when the bad event is likely. On a tool-lookup or a customer-support resolve — where one good line of reasoning almost always works — ToT is pure premium with no claim. On Game-of-24, where the first

guess is usually wrong, the $16\times$ premium buys a real jump in solve rate. Same machinery, opposite verdict, decided by the base failure rate.

7 The search tree, drawn



One root, $b = 3$ children, $b^2 = 9$ grandchildren: $1 + 3 + 9 = 13$ nodes shown (that is $D = 2$). Add the 27 great-grandchildren of $D = 3$ and you reach 40. The picture is Equation (3): every extra level multiplies the frontier by b . ReAct, by contrast, would be a single straight chain down the left edge — K nodes, no fan-out.

8 Search-cost cheat-sheet

ReAct calls (single path)	$\text{calls} = K$
Nodes at level ℓ	b^ℓ
Total nodes to depth D	$N(b, D) = \frac{b^{D+1} - 1}{b - 1}$
ToT unpruned calls (gen + eval)	$\approx 2 N(b, D)$
ToT beam-pruned calls	$\approx w b D$
Cost per call	$c_{\text{call}} = T_{\text{in}}p_{\text{in}} + T_{\text{out}}p_{\text{out}}$
Dominant growth (unpruned)	$O(b^D)$ — exponential in depth
Beam special case	search complete only if winner stays in top w

9 An honest word about these numbers

The token sizes ($T_{\text{in}} = 2000$, $T_{\text{out}} = 300$) and the flat per-call cost are illustrative round numbers, picked so the dollar sums are followable by hand. Real ToT calls vary: the generate call and the evaluate call are usually different sizes, deeper nodes carry more context, and many implementations batch the scoring. The “ $\times 2$ per node” and the flat c_{call} are deliberate simplifications. What is *not* illustrative is the structure: the unpruned node count is exactly the geometric series $\frac{b^{D+1}-1}{b-1}$, it grows like b^D , and beam pruning collapses it to a $w b D$ product by giving up completeness. Change the constants and the dollar figures move; the exponential-versus-linear shape does not. That shape is why the module rates ToT four cost-stars and calls it a research tool rather than a default.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the geometric series, the explosion table, the beam table, and the dollar comparison; change b , D , w , or the prices to explore your own search budget.

```

1  # tot_cost.py -- reproduces every number in "ReAct vs Tree-of-Thoughts".
2  P_IN, P_OUT = 3e-6, 15e-6 # $ per token ($3 / $15 per 1M)
3  T_IN, T_OUT = 2000, 300 # flat tokens read / written per LLM call
4  c_call = T_IN * P_IN + T_OUT * P_OUT # = $0.0105
5
6  def react_calls(K): # single path: K steps = K calls (linear)
7      return K
8
9  def tot_nodes(b, D): # geometric series (b^(D+1)-1)/(b-1)
10     return (b**(D + 1) - 1) // (b - 1)
11
12 def tot_beam(w, b, D): # beam-pruned: ~ w*b*D calls
13     return w * b * D
14
15 # --- geometric series matches the brute-force sum ---
16 for b, D in [(3, 3), (2, 3), (3, 4)]:
17     n = tot_nodes(b, D)
18     brute = sum(b**l for l in range(D + 1))
19     print(f"b={b} D={D}: closed={n} sum={brute} match={n == brute}")
20
21 # --- explosion + beam table ---
22 print(" b D | ReAct unpruned(2N) beam(w=2)")
23 for b, D in [(2,2),(2,3),(3,2),(3,3),(3,4),(4,3)]:
24     K = D + 2
25     print(f" {b} {D} | {react_calls(K):5d} {2*tot_nodes(b,D):13d} {tot_beam(2,b,D):11d}")
26
27 # --- dollars at headline b=3, D=3 ---
28 print(f"\ncost per call = ${c_call:.4f}")
29 for name, calls in [("ReAct", 5),
30                     ("ToT beam w=2", tot_beam(2, 3, 3)),
31                     ("ToT unpruned", 2 * tot_nodes(3, 3))]:
32     print(f"{name:14s} calls={calls:3d} cost=${calls*c_call:.4f}")
33
34 # Expected output:
35 # b=3 D=3: closed=40 sum=40 match=True
36 # b=2 D=3: closed=15 sum=15 match=True
37 # b=3 D=4: closed=121 sum=121 match=True
38 # b D | ReAct unpruned(2N) beam(w=2)
39 # 2 2 | 4 14 8
40 # 2 3 | 5 30 12
41 # 3 2 | 4 26 12
42 # 3 3 | 5 80 18
43 # 3 4 | 6 242 24
44 # 4 3 | 5 170 24
45 #
46 # cost per call = $0.0105
47 # ReAct calls= 5 cost=$0.0525
48 # ToT beam w=2 calls= 18 cost=$0.1890
49 # ToT unpruned calls= 80 cost=$0.8400

```

— end of document —